COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	9/17/2026	**DATE SUBMITTED:**	9/20/2026
NAME:	Dimaguiba, Lorenzo James M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 9.1 TREE ADT

1. PROCEDURE OUTPUT

TASK 1:

CODE

MAIN:

TREEEEEEE > PRO > mainPRO.cpp > ...

```cpp
#include <iostream>
#include "genBST.h"

int main(){

    //lvl 0
    GeneralTree<char>* root = new GeneralTree('A');

    //level 1
    root->insertFirst('B');
    root->insertSecond('C');
    root->insertThird('D');
    root->insertFourth('E');
    root->insertFifth('F');
    root->insertSixth('G');

    //level 2
    root->getThirdChild()->insertFirst('H');

    root->getFourthChild()->insertFirst('I');
    root->getFourthChild()->insertSecond('J');

    root->getFifthChild()->insertFirst('K');
    root->getFifthChild()->insertSecond('L');
    root->getFifthChild()->insertThird('M');

    root->getSixthChild()->insertFirst('N');

    //level 3
    root->getFourthChild()->getSecondChild()->insertFirst('P');
    root->getFourthChild()->getSecondChild()->insertSecond('Q');

    //traverse
    std::cout<<"Preorder Order: ";
    root->preOrder();

    std::cout<<"\nPostorder Order: ";
    root->postOrder();

    std::cout<<"\nInorder Order: ";
    root->inOrder();




    return 0;
}

```

HEADER:

TREEEEEEE > PRO > C genBST.h > GeneralTree<T>

```h
#ifndef GENBST_H
#define GENBST_H
#include <iostream>

template<typename T>
class GeneralTree{

    private:
    T key;
    GeneralTree<T>* first;
    GeneralTree<T>* second;
    GeneralTree<T>* third;
    GeneralTree<T>* fourth;
    GeneralTree<T>* fifth;
    GeneralTree<T>* sixth;

    public:
    //constructor
    GeneralTree(T value);

    //insertion
    void insertFirst(T value);
    void insertSecond(T value);
    void insertThird(T value);
    void insertFourth(T value);
    void insertFifth(T value);
    void insertSixth(T value);

    //get positions
    T getRootVal();
    GeneralTree<T>* getFirstChild();
    GeneralTree<T>* getSecondChild();
    GeneralTree<T>* getThirdChild();
    GeneralTree<T>* getFourthChild();
    GeneralTree<T>* getFifthChild();
    GeneralTree<T>* getSixthChild();

    //traversal
    void preOrder();
    void postOrder();
    void inOrder();

};

template <typename T>
GeneralTree<T>::GeneralTree(T value){
    key = value;
    first = nullptr;
    second = nullptr;
    third = nullptr;
    fourth = nullptr;
    fifth = nullptr;
    sixth = nullptr;
}
template <typename T>
T GeneralTree<T>::getRootVal(){ //for testing purposes
    return key;
}
```

```cpp
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::getFirstChild(){
    return first;
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::getSecondChild(){
    return second;
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::getThirdChild(){
    return third;
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::getFourthChild(){
    return fourth;
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::getFifthChild(){
    return fifth;
}

template <typename T>
GeneralTree<T>* GeneralTree<T>::getSixthChild(){
    return sixth;
}

template <typename T>
void GeneralTree<T>::insertFirst(T newNode){

    //if null
    if(first == nullptr){
        first = new GeneralTree<T>(newNode);
    }

    //if ever we want to add a node in a specific place
    else{
        GeneralTree<T>* newChild = new GeneralTree<T>(newNode);

        //move existing child below new node
        newChild->first = first;

        //make new node the child;
        first = newChild;

    }
}

```

```cpp
template <typename T>
void GeneralTree<T>::insertSecond(T newNode){

    //if null
    if(second == nullptr){
        second = new GeneralTree<T>(newNode);
    }

    //if ever we want to add a node in a specific place
    else{
        GeneralTree<T>* newChild = new GeneralTree<T>(newNode);

        //move existing child below new node
        newChild->second = second;

        //make new node the child;
        second = newChild;

    }
}

template <typename T>
void GeneralTree<T>::insertThird(T newNode){

    //if null
    if(third == nullptr){
        third = new GeneralTree<T>(newNode);
    }

    //if ever we want to add a node in a specific place
    else{
        GeneralTree<T>* newChild = new GeneralTree<T>(newNode);

        //move existing child below new node
        newChild->third = third;

        //make new node the child;
        third = newChild;

    }
}

template <typename T>
void GeneralTree<T>::insertFourth(T newNode){

    //if null
    if(fourth == nullptr){
        fourth = new GeneralTree<T>(newNode);
    }

    //if ever we want to add a node in a specific place
    else{
        GeneralTree<T>* newChild = new GeneralTree<T>(newNode);

        //move existing child below new node
        newChild->fourth = fourth;

```

```
168             //make new node the child;
169             fourth = newChild;
170 
171         }
172     }
173 
174     template <typename T>
175     void GeneralTree<T>::insertFifth(T newNode){
176 
177         //if null
178         if(fifth == nullptr){
179             fifth = new GeneralTree<T>(newNode);
180         }
181 
182         //if ever we want to add a node in a specific place
183         else{
184             GeneralTree<T>* newChild = new GeneralTree<T>(newNode);
185 
186             //move existing child below new node
187             newChild->fifth = fifth;
188 
189             //make new node the child;
190             fifth = newChild;
191 
192         }
193     }
194 
195     template <typename T>
196     void GeneralTree<T>::insertSixth(T newNode){
197 
198         //if null
199         if(sixth == nullptr){
200             sixth = new GeneralTree<T>(newNode);
201         }
202 
203         //if ever we want to add a node in a specific place
204         else{
205             GeneralTree<T>* newChild = new GeneralTree<T>(newNode);
206 
207             //move existing child below new node
208             newChild->sixth = sixth;
209 
210             //make new node the child;
211             sixth = newChild;
212 
213         }
214     }
215 
```

Task 2:

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Note: Leaf nodes have height = 0.

Task 3:

Pre-order	A B C D H E I J P Q F K L M G N
Post-order	B C H D I P Q J E K L M F N G A
In-order	B A C H D I E P J Q K F L M N G

Task 3.2:

CODE

```
216  template <typename T>
217  void GeneralTree<T>::preOrder(){
218
219      //Order:
220      // root->first.....->sixth
221      std::cout<< key << " ";
222
223      if(first != nullptr){
224          first->preOrder();
225      }
226
227      if(second != nullptr){
228          second->preOrder();
229      }
230
231      if(third != nullptr){
232          third->preOrder();
233      }
234
235      if(fourth != nullptr){
236          fourth->preOrder();
237      }
238
239      if(fifth != nullptr){
240          fifth->preOrder();
241      }
242
243      if(sixth != nullptr){
244          sixth->preOrder();
245      }
246
247
248  }
249
250  template <typename T>
251  void GeneralTree<T>::postOrder(){
252
253      //Order:
254      // first.....->sixth->root
255
256      if(first != nullptr){
257          first->postOrder();
258      }
259
260      if(second != nullptr){
261          second->postOrder();
262      }
263
264      if(third != nullptr){
265          third->postOrder();
266      }
267
268      if(fourth != nullptr){
269          fourth->postOrder();
270      }
271
272      if(fifth != nullptr){
273          fifth->postOrder();
274      }
275
276      if(sixth != nullptr){
277          sixth->postOrder();
278      }
279
280      std::cout<< key << " ";
281
282  }
```

```
283
284  template <typename T>
285  void GeneralTree<T>::inOrder(){
286
287      //Order:
288      // first->root->......->sixt
289
290      if(first != nullptr){
291          first->inOrder();
292      }
293
294      std::cout<< key << " ";
295
296      if(second != nullptr){
297          second->inOrder();
298      }
299
300      if(third != nullptr){
301          third->inOrder();
302      }
303
304      if(fourth != nullptr){
305          fourth->inOrder();
306      }
307
308      if(fifth != nullptr){
309          fifth->inOrder();
310      }
311
312      if(sixth != nullptr){
313          sixth->inOrder();
314      }
315  }
316
317  #endif
```

Task 3.3:

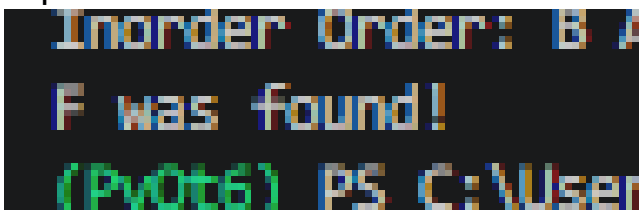
CODE

```
433  template <typename T>
434  void GeneralTree<T>::findData(int choice, T search){
435
436      switch(choice){
437          case 1:
438              preOrderSearch(search);
439              break;
440
441          case 2:
442              inOrderSearch(search);
443              break;
444
445          case 3:
446              postOrderSearch(search);
447              break;
448
449          default:
450              std::cout<<"Invalid Choice!"<<std::endl;
451              return;
452      }
453
454  }
455
456
457
458
```

```
43      std::cout<<std::endl;
44      root->findData(1, 'F'); //will not output anything if false
```

Note: In order to save space on this document, please just access it through the file.

Output:



Task 3.4:

```
// "O" activity
root->getSixthChild()->insertSecond('O');
```

```
root->findData(2, 'O'); //will not output anything if false
```

```
Preorder Order: A B C D H E I J P Q F K L M G N O
Postorder Order: B C H D I P Q J E K L M F N O G A
Inorder Order: B A C H D I E P J Q K F L M N G O
F was found!
O was found!
```

Observation: It was added into the tree, order sort added "O" and also found using search algorithms

Observation/Analysis:

There are different types of trees each with their own structures and use cases. In order to add to a tree, one must begin from its root. Since a tree is a collection of nodes, we make use of pointers to navigate and add new nodes, branches and leaves. The general tree can have an "N" number of children compared to other types. There are 3 types of traversal, in order, that begins from the root, then from left to right; inorder begins on the left-most side, then its root, to the rest of the elements, finally the postorder traversal starts with all children nodes before the root. Depth refers to how deep a node is from the root, and the height is the level at which it begins from a leaf on the bottom. Leaves have a height of zero and depth may vary.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

CODE

MAIN:

Dimaguiba_HOA9 > Supplementary > main_supp.cpp > main()

```cpp
#include <iostream>
#include "BST.h"
const int maxVal = 7;

int main(){

int inVal[maxVal] = {3, 9, 18, 0, 1, 4, 5};
BinarySearchTree<int>* tree = new BinarySearchTree(2);

    for (int i = 0; i < maxVal; i++){
        tree->insert(inVal[i]);

    }

    //optional
    /*
    tree->insert(3);
    tree->insert(9);
    tree->insert(18);
    tree->insert(0);
    tree->insert(1);
    tree->insert(4);
    tree->insert(5);
    */

    std::cout<<"Preorder output: ";
    tree->preOrder();
    std::cout<<std::endl;

    std::cout<<"Inorder output: ";
    tree->inOrder();
    std::cout<<std::endl;

    std::cout<<"Post output: ";
    tree->postOrder();
    std::cout<<std::endl;


    return 0;
}
```

HEADER:

Dimaguiba_HOA9 > Supplementary > C BST.h > ...

```
#ifndef BST_H
#define BST_H

template <typename T>
class BinarySearchTree{
    private:
    T key;
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;


    public:
    //constructor
    BinarySearchTree(T value);
    //insertion
    void insert(T value);
    //traversal
    void preOrder();
    void inOrder();
    void postOrder();
};

template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value){
    key = value;
    left = nullptr;
    right = nullptr;
}

template <typename T>
void BinarySearchTree<T>::insert(T value){

    //go left if value is < key
    if(value < key){

        if(left == nullptr){
            left = new BinarySearchTree<T>(value);
        }
        else{
            left->insert(value);

        }
    //go right if value > key
    }

    else if(value > key){
        if (right == nullptr){
            right = new BinarySearchTree<T>(value);
        }
        else{
            right->insert(value);
        }
    }

}

template <typename T>
void BinarySearchTree<T>::preOrder(){

    std::cout<< key << " ";

    if(left != nullptr){
        left->preOrder();
    }

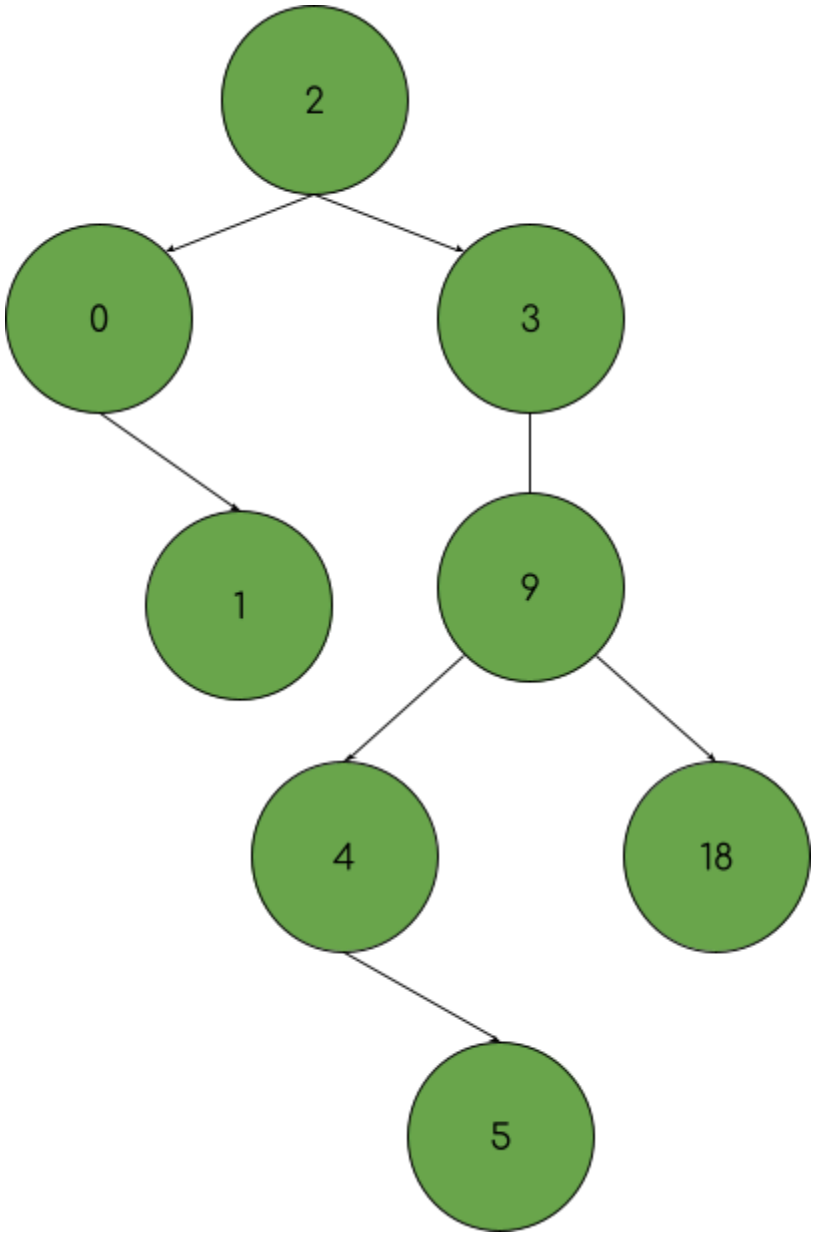
    if(right != nullptr){
        right->preOrder();
    }

```

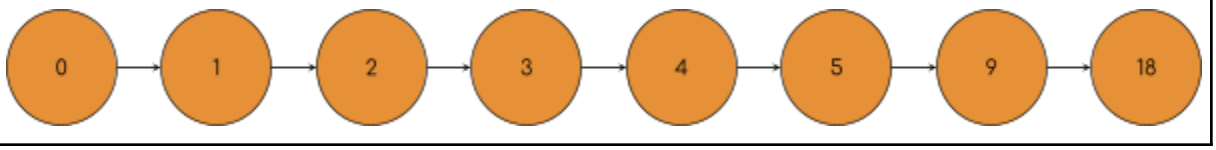
```
71  }
72
73  template <typename T>
74  void BinarySearchTree<T>::inOrder(){
75
76      if(left != nullptr){
77          left->inOrder();
78      }
79      std::cout<<key<<" ";
80
81      if(right != nullptr){
82          right->inOrder();
83      }
84
85  }
86
87  template <typename T>
88  void BinarySearchTree<T>::postOrder(){
89
90      if(left != nullptr){
91          left->postOrder();
92      }
93
94      if(right != nullptr){
95          right->postOrder();
96      }
97
98      std::cout<<key<<" ";
99
100 }
101
102
103
104
105 #endif
```

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.

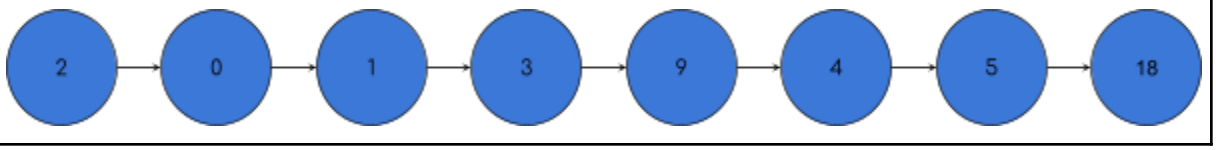
TREE DIAGRAM



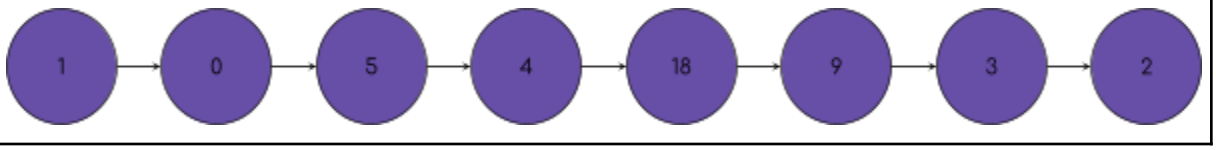
IN-ORDER (LEFT->ROOT->RIGHT)



PRE-ORDER (ROOT->LEFT->RIGHT)



POST-ORDER (LEFT->RIGHT->ROOT)



Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

OUTPUT

```
Preorder output: 2 0 1 3 9 4 5 18
Inorder output: 0 1 2 3 4 5 9 18
Post output: 1 0 5 4 18 9 3 2
(PyOt6) PS C:\Users\shosh\OneDrive\Documents\VSCODES>
```

Pre-order Traversal

```
58  template <typename T>
59  void BinarySearchTree<T>::preOrder(){
60
61      std::cout<< key << " ";
62
63      if(left != nullptr){
64          left->preOrder();
65      }
66
67      if(right != nullptr){
68          right->preOrder();
69      }
70
71  }
```

Output:

```
Preorder output: 2 0 1 3 9 4 5 18
```

Post-order Traversal

```
87  template <typename T>
88  void BinarySearchTree<T>::postOrder(){
89
90      if(left != nullptr){
91          left->postOrder();
92      }
93
94      if(right != nullptr){
95          right->postOrder();
96      }
97
98      std::cout<<key<<" ";
99
100 }
```

Output:

```
Post output: 1 0 5 4 18 9 3 2
```

In-order Traversal

```
73  template <typename T>
74  void BinarySearchTree<T>::inOrder(){
75
76      if(left != nullptr){
77          left->inOrder();
78      }
79      std::cout<<key<<" ";
80
81      if(right != nullptr){
82          right->inOrder();
83      }
84
85  }
```

Output:

```
Inorder output: 0 1 2 3 4 5 9 18
```

Conclusion/Analysis:

Binary Search Trees are trees with up to 2 children. Just like before, it runs on groups of nodes, subtrees, and edges all connected together. Depending on the root, the binary search tree encounters a number higher or lower than it. If it is higher, it places that value to its right node child, the lower the value, the other side (left). This goes on as it adds more nodes, creating subtrees, each with higher and lower values of their own. The way these new nodes are created is through the process of recursion. It calls the pointer of that node, and runs the same method on it. This applies to traversal as well. The output of traversal depends on whether it is in pre-order (root, left, then right), in-order(left, root, right) and post-order(left, right, root). They continue their traversal until they reach a leaf which ends one recursion process, allowing the rest of it to run (think of it as one long nested system).

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

- I would say that I found the binary search tree to be more simple to implement compared to the general tree. As they had more children to worry about. I was greatly confused with how I had to search if the element was found as well, but I managed to make it work. I based my procedure's traversal functions on the binary search tree, the main reference point being the root. However, for the life of me (I spent the most time here), trying to find a resource of how an In-order method would work in a general table. It always seemed to direct me to BSTs instead.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of Concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I was struggling to find a resource on how to apply in-order traversal on the general table. So I had to ask for a resource. The resource I found was so niche (the type you won't find on the front page of search engines) but it gave me enough information I need to only find out that there is no defined method. However, the common practice was as follows: left, root, then the rest.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.